Data Warehousing Project

Group Member:

Antonius Dimitri Adhisatrio

Student ID – 24253701

Nyan Tun Aye

Student ID – 23198368

SECTION A

1 Project Overview

In the current competitive and rapidly evolving digital landscape, effectively utilizing data has become an essential advantage for companies in every industry. OfficeProducts, a major supplier of office products in the country, showcases this need by merging a conventional retail model with an expanding online presence. The company has a network of physical stores across the country, but most of its sales now come from its online platform. This change has not only heightened the intricacy of its operations but also amplified the necessity for decisions based on data.

With a variety of client organizations, from small businesses to large enterprises, OfficeProducts needs to customize its services and offerings to address the different needs of its customers. The management of the company recognizes that examining client buying patterns can provide important insights. These insights can lead to strategic changes, like improving product delivery times, assessing competitive courier costs, and organizing targeted promotions that boost customer engagement and satisfaction.

A well-organized data flow is essential to back these strategic initiatives. Data from the data sources will be ETL-ed into the data warehouse, also operational and monitoring tools is also used throughout the data pipeline. Figure 1 will show us how the data flow works.

1.1 Data Flow

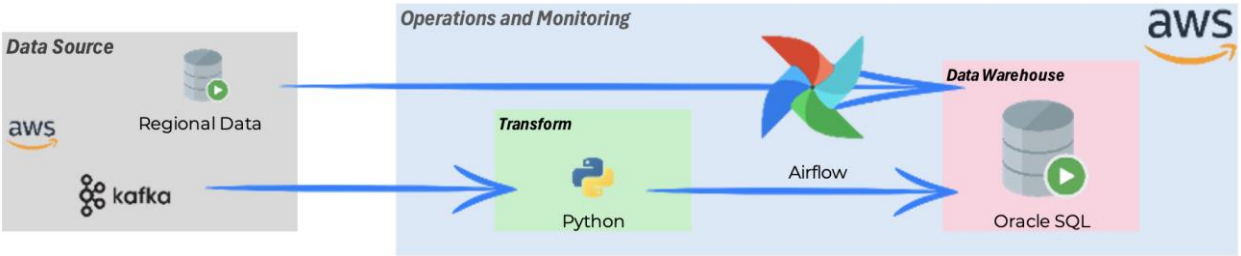

Figure 1. Overall Data Pipeline

These are the tools we use for managing data flow from the data source to the data warehouse. Given that OfficeProducts has several stores across the country, we must account for both online and offline transactions. Offline transaction data is processed through daily batch uploads to the data warehouse, while online transactions are transformed using Python and stored in the data warehouse in real time. Airflow is used to control and monitor all data jobs. Here is an explanation of the tools we use:

- Kafka
  Kafka serves as an enterprise service bus that transmits event-driven data. It provides real-time updates on any changes in the backend service. We configure Kafka topics for the backend services that need their data stored in the data warehouse

- Python
  Python is used to transform or normalize data received from Kafka before it flows into the data warehouse

- Airflow
  Airflow is utilized for managing dependencies across hundreds of data jobs. It also handles job scheduling, monitoring, and alerting

1.2 Data Warehouse Schema

Before we can fully utilize the data for reporting and analytics, we must first build a solid data warehouse schema as the foundation. When dealing with data—whether it's data flow or data quality—things can get quite complex, especially with online data. Issues such as duplicate records, missing data, or failed jobs can arise. This is why it's essential to segregate the data warehouse schema to address different purposes and ensure the data is structured properly for each use case. See Figure 2 for the data warehouse schema.

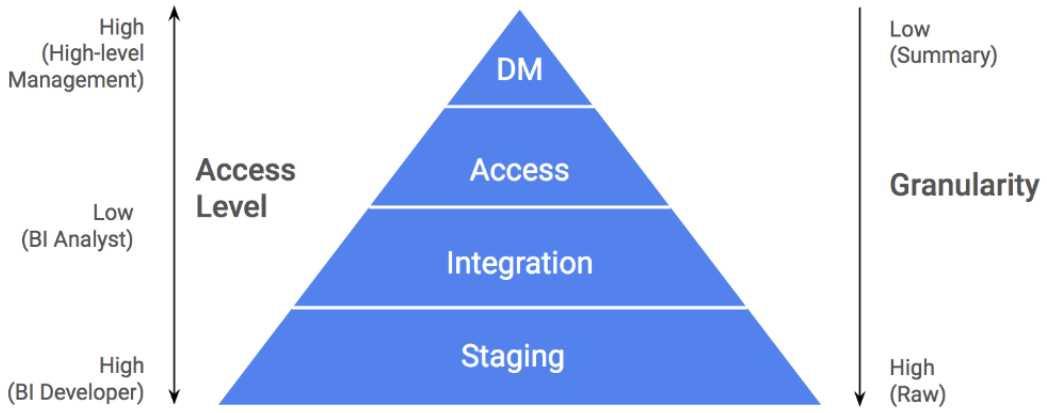

Figure 2. Data Warehouse Schemas

Here is an explanation for each data schema layer purposes:

- Staging
  The staging schema is where all data from the source is initially parked. As the first layer in the data warehouse, this schema contains raw, unprocessed data. Only minimal modifications or transformations are applied to ensure the data retains its original business value. Access to this schema is limited to data engineers.

- Integration
  The second layer stores integrated data from multiple tables in the Staging layer. In this layer, extensive modifications and transformations are performed to meet business requirements and create a cohesive data set. Access to this schema is limited to data engineers.

- Access
  The third layer contains data that is ready for user access. This schema includes summarized tables from the Integration layer and is designed for easy retrieval of data summaries on a daily, weekly, or monthly basis. It also houses the dimension tables, supporting quick, efficient querying. Access to this schema is openly available if they have access permission.

- DM – Data Mart

The final layer, the Data Mart, contains data tables specifically classified for reporting and analytics purposes. It stores data tailored for specific reports and serves as the primary reporting layer for high-level stakeholders in the company. Access to this schema is limited to specific business users.

2 Entity Relationship Diagram

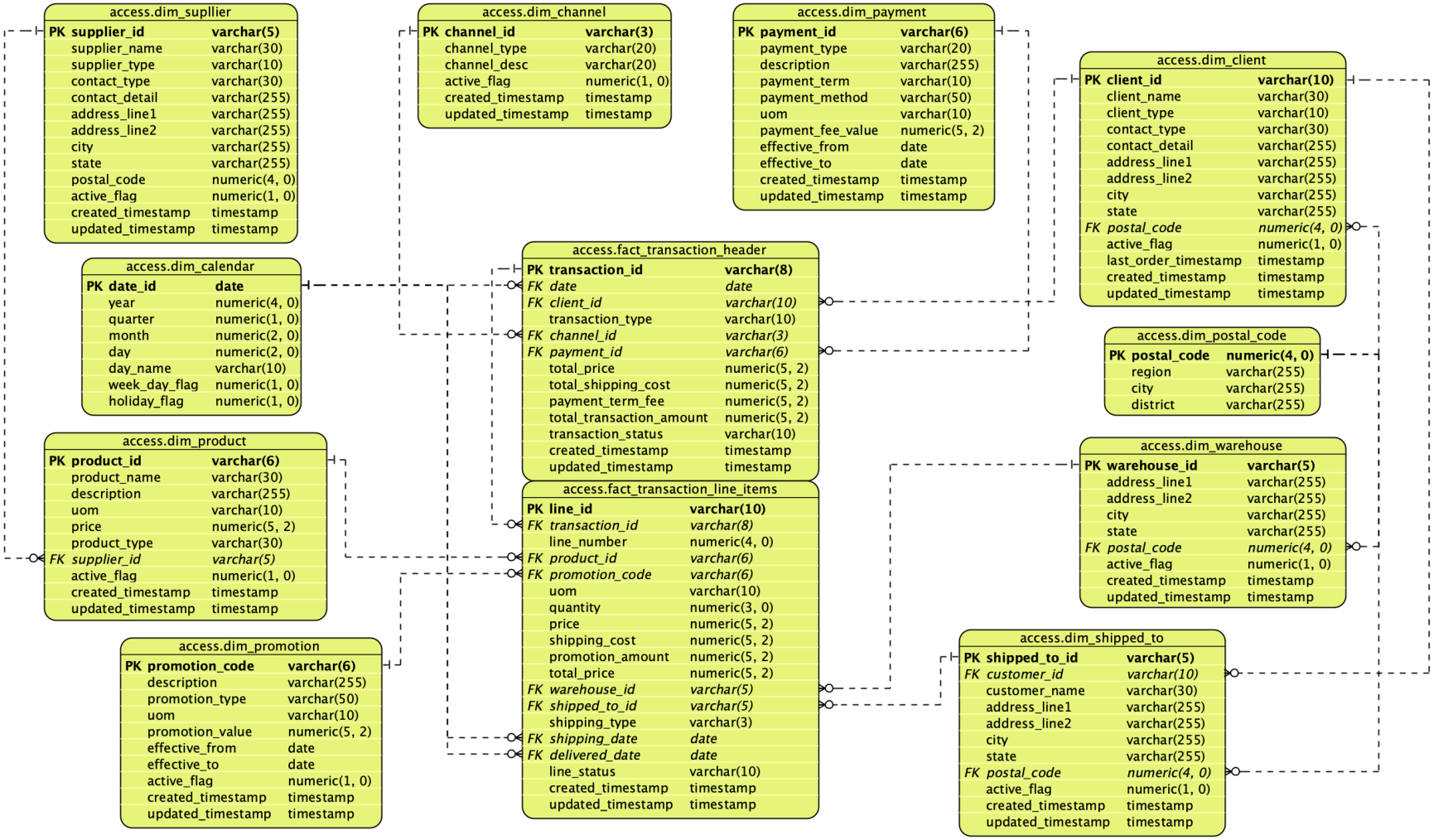

Figure 3. *access* Schema Entity Relationship Diagram for Sales Transactions

The primary objective of this report is to create an Entity Relationship Diagram (ERD) that supports analytical purposes in the sales domain, helping to inform and refine sales strategies, optimize selling methods, and enhance promotional planning. Figure 3 presents the complete ERD, including facts, dimensions, and attributes designed to serve these specific objectives.

These data are stored in the Access layer of our data warehouse schema. While some fact tables from other departments, such as logistics, are not displayed in the ERD, the fact tables that are shown are derived from multiple sources, including logistic data. We have retained some of the original column names from master data and data streams for ease of use.

The ERD was created using Visual Paradigm. From the ERD, we can see that each table (except **dim_calendar** and **dim_postal_code**) contains the same two columns name:

- **created_timestamp**
  This column captures the original timestamp when the transaction was created

- **updated_timestamp**
  This column captures the timestamp when a row is modified. By default, this value matches the created_timestamp unless changes are made

These timestamps help track of changes to the data or identify errors in data processing, ensuring any delays in data visibility are accounted for.

With this ERD, we can do several analyses, such as:

- **Client Retention and Engagement**
  We can perform cohort analysis to determine the last time each client made an order, enabling us to engage them and encourage future transactions

- **Transaction Channel Optimization**
  By analyzing channel identifiers, we can identify which channels perform best and explore ways to optimize other channels

- **Day-by-Day Analysis**
  Normal working days will be business as usual, but since there are many online transactions, we can also analyze transaction activity on weekends and public holidays, providing an additional perspective for developing a strategy. Additionally, we can identify which days transactions peak, whether it’s Monday, Tuesday, or any other days.

- **Optimizing Similar Product Sales**
  By examining product types, we can analyze which similar products perform best and identify ways to improve the performance of underperforming products

- **Partnership with Payment Provider**
  Analyzing popular payment channels can provide opportunities for partnerships with payment providers to run joint promotions

- **Payment method strategy**
  OfficeProducts has many corporate clients, so we can develop a strategy to encourage large orders with the option of installment payments. This will attract clients to make transactions, while keep maintaining their cashflows. Also, it will add up more revenue, since paying with installments have payment fees.

- **Client Growth**
  Using the *created_timestamp*, we can track when a client was added on our platform, providing insight into new client growth over time

- **Shipping Cost Optimization**
  With information on where items are shipped from (warehouse) and the client's delivery address, we can estimate shipping costs and optimize delivery routes

- **Promotion Strategy**
  By analyzing promotion data, we can evaluate the success of promotions, such as how many times promotion codes have been used since its effective date and how much sales generated using the promotion codes

- **Delivery Service Level Agreement (SLA)**
  We can analyze our delivery SLA to determine whether we have breached it with any clients, which could potentially result in losing them

Although we used Visual Paradigm to create the ERD, it does not natively support Oracle SQL-specific attributes. As a result, some attributes are shown in a more general SQL format. To address this, the following data dictionary section defines the attributes specifically for an Oracle SQL database.

3 Data Dictionary

- **access.fact_transaction_header**

Column Name	Data Type	Nullable	Key	Description
transaction_id	VARCHAR2(8)	No	Primary Key	Unique identifier for transactions
transaction_date	DATE	No	Foreign Key	Date of the transaction
client_id	VARCHAR2(10)	No	Foreign Key	Identifier of the client related to the transaction
transaction_type	VARCHAR2(10)	No		Type of the transaction (e.g., standard, blanket)
channel_id	VARCHAR2(3)	No	Foreign Key	Identifier of the channel through which the transaction was made
payment_id	VARCHAR2(6)	No	Foreign Key	Identifier of the payment type used in the transaction
transaction_amount	NUMBER(5, 2)	No		Total amount of the product(s) transaction
total_shipping_cost	NUMBER(5, 2)	No		Total shipping cost for the transaction. Default 0.00
payment_term_fee	NUMBER(5, 2)	No		Fee associated with the payment term. Default 0.00
total_transaction_amount	NUMBER(5, 2)	No		Total transaction amount including fees. Default 0.00
transaction_status	VARCHAR2(10)	No		Status of the transaction (e.g., ordered, processed, completed)
created_timestamp	TIMESTAMP	No		Timestamp of when the transaction record was created
updated_timestamp	TIMESTAMP	No		Timestamp of when the transaction record was last updated

- **access.fact_transaction_line_items**

Column Name	Data Type	Nullable	Key	Description
line_id	VARCHAR2(10)	No	Primary Key	Unique identifier for transaction line-items
transaction_id	VARCHAR2(8)	No	Foreign Key	Identifier of the transaction to which the line-item belongs
line_number	NUMBER(4, 0)	No		Line number of the transaction item (e.g. 0001, 0002)
product_id	VARCHAR2(6)	No	Foreign Key	Identifier of the product in the line-item
promotion_code	VARCHAR2(6)	Yes	Foreign Key	Discount code applied to the line-item (if any)
uom	VARCHAR2(10)	No		Unit of measure for the product in the line-item (e.g. Piece, lot, Kilogram, Gram)
quantity	NUMBER(3, 0)	No		Quantity of the product in the line-item. Default 1

price	NUMBER(5, 2)	No		Price of the product in the line-item
shipping_cost	NUMBER(5, 2)	No		Shipping cost for the line-item
promotion_amount	NUMBER(5, 2)	No		Promotion amount for the line-item. Default 0.00
total_price	NUMBER(5, 2)	No		Total price for the line-item
warehouse_id	VARCHAR2(5)	No	Foreign Key	Identifier of the warehouse related to the line item
shipped_to_id	VARCHAR2(5)	No	Foreign Key	Identifier of the location where the item is shipped to
shipping_type	VARCHAR2(3)	No		Type of shipping for the line-item
shipping_date	DATE	Yes		Date of when line-item was shipped (if any)
delivered_date	DATE	Yes		Date of when line-item was delivered (if any)
line_status	VARCHAR2(10)	No		Status of the line-item (e.g. ordered, processed, shipped, delivered, completed)
created_timestamp	TIMESTAMP	No		Timestamp of when the line-item record was created
updated_timestamp	TIMESTAMP	No		Timestamp of when the line-item record was last updated

- **access.dim_clients**

Column Name	Data Type	Nullable	Key	Description
client_id	VARCHAR2(10)	No	Primary Key	Unique identifier for clients
client_name	VARCHAR2(30)	No		Name of the client
client_type	VARCHAR2(10)	No		Type of client (e.g., individual, corporate)
contact_type	VARCHAR2(30)	No		Type of contact (e.g., phone, email)
contact_detail	VARCHAR2(255)	No		Contact details of the client
address_line1	VARCHAR2(255)	No		Primary address line of the client
address_line2	VARCHAR2(255)	Yes		Secondary address line of the client (if needed)
city	VARCHAR2(255)	No		City where the client is located
state	VARCHAR2(255)	No		State where the client is located
postal_code	NUMBER(4, 0)	No	Foreign Key	Postal code of the client
active_flag	NUMBER(1, 0)	No		Indicates if the client is active (1) or inactive (0)
last_order_timestamp	TIMESTAMP	Yes		Timestamp of the last order made by the client
created_timestamp	TIMESTAMP	No		Timestamp of when the record was created
updated_timestamp	TIMESTAMP	No		Timestamp of when the record was last updated

- ## access.dim_product

Column Name	Data Type	Nullable	Key	Description
product_id	VARCHAR2(10)	No	Primary Key	Unique identifier for products
product_name	VARCHAR2(30)	No		Name of the product
description	VARCHAR2(255)	Yes		Description of the product
uom	VARCHAR2(10)	No		Unit of measure for the product (e.g. Piece, lot, Kilogram, Gram)
price	NUMBER(5, 2)	No		Price of the product
product_type	VARCHAR2(30)	No		Type of the product (e.g., electronic, furniture, stationary)
supplier_id	VARCHAR2(5)	No	Foreign Key	Identifier for the product's supplier
active_flag	NUMBER(1, 0)	No		Indicates if the product is active (1) or inactive (0)
created_timestamp	TIMESTAMP	No		Timestamp of when the record was created
updated_timestamp	TIMESTAMP	No		Timestamp of when the record was last updated

- ## access.dim_supplier

Column Name	Data Type	Nullable	Key	Description
supplier_id	VARCHAR2(5)	No	Primary Key	Unique identifier for suppliers
supplier_name	VARCHAR2(30)	No		Name of the supplier
supplier_type	VARCHAR2(10)	No		Type of supplier (e.g., retail, wholesale)
contact_type	VARCHAR2(30)	No		Type of contact (e.g., email, phone)
contact_detail	VARCHAR2(255)	No		Contact details of the supplier
address_line1	VARCHAR2(255)	No		Primary address line of the supplier
address_line2	VARCHAR2(255)	Yes		Secondary address line of the supplier (if any)
city	VARCHAR2(255)	No		City where the supplier is located
state	VARCHAR2(255)	No		State where the supplier is located
postal_code	NUMBER(4, 0)	No	Foreign Key	Postal code of the supplier
active_flag	NUMBER(1, 0)	No		Indicates if the supplier is active (1) or inactive (0)
created_timestamp	TIMESTAMP	No		Timestamp of when the record was created
updated_timestamp	TIMESTAMP	No		Timestamp of when the record was last updated

- ## access.dim_channel

Column Name	Data Type	Nullable	Key	Description
channel_id	VARCHAR2(5)	No	Primary Key	Unique identifier for channels
channel_type	VARCHAR2(20)	No		Type of channel (e.g., online, in-store, event)
channel_desc	VARCHAR2(20)	No		Description of the channel

active_flag	NUMBER(1, 0)	No		Indicates if the channel is active (1) or inactive (0)
created_timestamp	TIMESTAMP	No		Timestamp of when the record was created
updated_timestamp	TIMESTAMP	No		Timestamp of when the record was last updated

- ## access.dim_payment

Column Name	Data Type	Nullable	Key	Description
payment_id	VARCHAR2(6)	No	Primary Key	Unique identifier for payment types
payment_type	VARCHAR2(20)	No		Type of payment (e.g., online, in-store, installment, etc.)
description	VARCHAR2(255)	Yes		Description of the payment type
payment_term	VARCHAR2(10)	No		Payment terms associated with the payment type (e.g. cash, n30, n60, n90, etc.)
payment_method	VARCHAR2(50)	No		Method used for the payment (e.g. cash, cheque, payment gateway, etc.)
uom	VARCHAR2(10)	Yes		Unit of measure associated with the payment fee (e.g. percentage, price, etc.)
payment_fee_value	NUMBER(5, 2)	Yes		Fee value or amount associated with the payment
effective_from	DATE	Yes		Start date of the payment term's validity
effective_to	DATE	Yes		End date of the payment term's validity
created_timestamp	TIMESTAMP	No		Timestamp of when the record was created
updated_timestamp	TIMESTAMP	No		Timestamp of when the record was last updated

- ## access.dim_promotion

Column Name	Data Type	Nullable	Key	Description
promotion_code	VARCHAR2(6)	No	Primary Key	Unique identifier for discounts
description	VARCHAR2(255)	Yes		Description of the discount
promotion_type	VARCHAR2(50)	No		Type of discount (e.g., regular, referral, seasonal, event)
uom	VARCHAR2(10)	No		Unit of measure for the discount (e.g. percent, price)
promotion_value	NUMBER(5, 2)	No		Percentage value or amount of the discount
effective_from	DATE	Yes		Start date of the discount's validity (if any)
effective_to	DATE	Yes		End date of the discount's validity (if any)

active_flag	NUMBER(1, 0)	No		Indicates if the discount is active (1) or inactive (0)
created_timestamp	TIMESTAMP	No		Timestamp of when the discount record was created
updated_timestamp	TIMESTAMP	No		Timestamp of when the discount record was last updated

- ## access.dim_warehouse

Column Name	Data Type	Nullable	Key	Description
warehouse_id	VARCHAR2(5)	No	Primary Key	Unique identifier for warehouses
address_line1	VARCHAR2(255)	No		Primary address line of the warehouse
address_line2	VARCHAR2(255)	Yes		Secondary address line of the warehouse (if needed)
city	VARCHAR2(255)	No		City where the warehouse is located
state	VARCHAR2(255)	No		State where the warehouse is located
postal_code	NUMBER(4, 0)	No	Foreign Key	Postal code of the warehouse
active_flag	NUMBER(1, 0)	No		Indicates if the warehouse is active (1) or inactive (0)
created_timestamp	TIMESTAMP	No		Timestamp of when the record was created
updated_timestamp	TIMESTAMP	No		Timestamp of when the record was last updated

- ## access.dim_shipped_to

Column Name	Data Type	Nullable	Key	Description
shipped_to_id	VARCHAR2(5)	No	Primary Key	Unique identifier for shipping destinations
customer_id	VARCHAR2(10)	No	Foreign Key	Identifier of the customer related to the shipping destination
customer_name	VARCHAR2(30)	No		Name of the customer for shipping
address_line1	VARCHAR2(255)	No		Primary address line of the shipping destination
address_line2	VARCHAR2(255)	Yes		Secondary address line of the shipping destination (if needed)
city	VARCHAR2(255)	No		City of the shipping destination
state	VARCHAR2(255)	No		State of the shipping destination
postal_code	NUMBER(4, 0)	No	Foreign Key	Postal code of the shipping destination
active_flag	NUMBER(1, 0)	No		Indicates if the shipping destination is active (1) or inactive (0)

created_timestamp	TIMESTAMP	No		Timestamp of when the record was created
updated_timestamp	TIMESTAMP	No		Timestamp of when the record was last updated

- ## access.dim_postal code

Column Name	Data Type	Nullable	Key	Description
postal_code	NUMBER(4, 0)	No	Primary Key	Unique postal code
region	VARCHAR2(255)	No		Region associated with the postal code
city	VARCHAR2(255)	No		City associated with the postal code
district	VARCHAR2(255)	No		District associated with the postal code

- ## access.dim_calendar

Column Name	Data Type	Nullable	Key	Description
date_id	DATE	No	Primary Key	Unique date identifier
year	NUMBER(4, 0)	No		Year of the date
quarter	NUMBER(1, 0)	No		Quarter of the year (1-4)
month	NUMBER(2, 0)	No		Month of the year (1-12)
day	NUMBER(2, 0)	No		Day of the month (1-31)
day_name	VARCHAR2(10)	No		Name of the day (e.g., Monday)
week_day_flag	NUMBER(1, 0)	No		Flag indicating if the day is a weekday (1) or weekend (0)
holiday_flag	NUMBER(1, 0)	No		Flag indicating if the day is a holiday (1) or not (0)

4 Table Creation

Since we do not have access permission to create a new schema, so we can assume that these tables were created in the ***access*** schema. The following are screenshots from the Oracle SQL table creations:

- **access.dim_postal_code** table creation

Worksheet | Query Builder

```sql
/*
create access.dim_postal_code table
*/
CREATE TABLE dim_postal_code (
    postal_code NUMBER(4, 0) PRIMARY KEY,
    region VARCHAR2(255) NOT NULL,
    city VARCHAR2(255) NOT NULL,
    district VARCHAR2(255) NOT NULL
);
SELECT
    *
FROM
    dim_postal_code
```

Script Output | Query Result

SQL | All Rows Fetched: 0 in 0.086 seconds

POSTAL_CODE	REGION	CITY	DISTRICT

- **access.dim_calendar** table creation

Worksheet | Query Builder

```sql
/*
create access.dim_calendar table
*/
CREATE TABLE dim_calendar (
    date_id DATE PRIMARY KEY,
    year NUMBER(4, 0) NOT NULL,
    quarter NUMBER(1, 0) NOT NULL,
    month NUMBER(2, 0) NOT NULL,
    day NUMBER(2, 0) NOT NULL,
    day_name VARCHAR2(10) NOT NULL,
    week_day_flag NUMBER(1, 0) NOT NULL,
    holiday_flag NUMBER(1, 0) NOT NULL
);
SELECT
    *
FROM
    dim_calendar
```

Script Output | Query Result

SQL | All Rows Fetched: 0 in 0.051 seconds

DATE_ID	YEAR	QUARTER	MONTH	DAY	DAY_NAME	WEEK_DAY_FLAG	HOLIDAY_FLAG

- **access.dim_channel** table creation

Worksheet | Query Builder

```sql
/*
create access.dim_channel table
*/
CREATE TABLE dim_channel (
    channel_id VARCHAR2(3) PRIMARY KEY,
    channel_type VARCHAR2(20) NOT NULL,
    channel_desc VARCHAR2(255) NOT NULL,
    active_flag NUMBER(1, 0) DEFAULT 1 NOT NULL,
    created_timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,
    updated_timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL
);
SELECT
    *
FROM
    dim_channel
```

Script Output | Query Result

SQL | All Rows Fetched: 0 in 0.055 seconds

CHANNEL_ID	CHANNEL_TYPE	CHANNEL_DESC	ACTIVE_FLAG	CREATED_TIMESTAMP	UPDATED_TIMESTAMP

- **access.dim_payment** table creation

Worksheet | Query Builder

```sql
/*
create access.dim_payment table
*/
CREATE TABLE dim_payment (
    payment_id VARCHAR2(6) PRIMARY KEY,
    payment_type VARCHAR2(20) NOT NULL,
    description VARCHAR2(255),
    payment_term VARCHAR2(50) NOT NULL,
    payment_method VARCHAR2(10) NOT NULL,
    uom VARCHAR2(10) NOT NULL,
    payment_fee_value NUMBER(5, 2) NOT NULL,
    effective_from DATE,
    effective_to DATE,
    created_timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,
    updated_timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL
);
SELECT
    *
FROM
    dim_payment
```

Script Output | Query Result

SQL | All Rows Fetched: 0 in 0.059 seconds

PAYME...	PAYME...	DESCRI...	PAYME...	PAYME...	UOM	PAYME...	EFFEC...	EFFEC...	CREAT...	UPDAT...

- **access.dim_supplier** table creation

Worksheet | Query Builder

```sql
/*
create access.dim_supplier table
*/
CREATE TABLE dim_supplier (
    supplier_id VARCHAR2(5) PRIMARY KEY,
    supplier_name VARCHAR2(30) NOT NULL,
    supplier_type_type VARCHAR2(10) NOT NULL,
    contact_type VARCHAR2(30) NOT NULL,
    contact_detail VARCHAR2(255) NOT NULL,
    address_line1 VARCHAR2(255) NOT NULL,
    address_line2 VARCHAR2(255),
    city VARCHAR2(255) NOT NULL,
    state VARCHAR2(255) NOT NULL,
    postal_code NUMBER(4, 0) NOT NULL,
    active_flag NUMBER(1, 0) DEFAULT 1 NOT NULL,
    created_timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,
    updated_timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL
);
SELECT
    *
FROM
    dim_supplier
```

Script Output | Query Result

SQL | All Rows Fetched: 0 in 0.07 seconds

SUPPLI...	SUPPLI...	SUPPLI...	CONT...	CONT...	ADDRE...	ADDRE...	CITY	STATE	POSTA...	ACTIV...	CREAT...	UPDAT...

- **access.dim_client** table creation

Worksheet | Query Builder

```
/*
create access.dim_client table
*/
CREATE TABLE dim_client (
    client_id VARCHAR2(10) PRIMARY KEY,
    client_name VARCHAR2(30) NOT NULL,
    client_type VARCHAR2(10) NOT NULL,
    contact_type VARCHAR2(30) NOT NULL,
    contact_detail VARCHAR2(255) NOT NULL,
    address_line1 VARCHAR2(255) NOT NULL,
    address_line2 VARCHAR2(255),
    city VARCHAR2(255) NOT NULL,
    state VARCHAR2(255) NOT NULL,
    postal_code NUMBER(4, 0) NOT NULL,
    active_flag NUMBER(1, 0) DEFAULT 1 NOT NULL,
    last_order_timestamp TIMESTAMP,
    created_timestamp TIMESTAMP NOT NULL,
    updated_timestamp TIMESTAMP NOT NULL,
    FOREIGN KEY (postal_code) REFERENCES dim_postal_code(postal_code)
);
SELECT
    *
FROM
    dim_client
```

Script Output | Query Result

SQL | All Rows Fetched: 0 in 0.059 seconds

CLIENT...	CLIENT...	CLIENT...	CONT...	CONT...	ADDRE...	ADDRE...	CITY	STATE	POSTA...	ACTIV...	LAST_...	CREAT...	UPDAT...

- **access.dim_product** table creation

Worksheet | Query Builder

```
/*
create access.dim_product table
*/
CREATE TABLE dim_product (
    product_id VARCHAR2(6) PRIMARY KEY,
    product_name VARCHAR2(30) NOT NULL,
    description VARCHAR2(255),
    uom VARCHAR2(10) NOT NULL,
    price NUMBER(5, 2) NOT NULL,
    product_type VARCHAR2(30) NOT NULL,
    supplier_id VARCHAR2(5) NOT NULL,
    active_flag NUMBER(1, 0) DEFAULT 1 NOT NULL,
    created_timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,
    updated_timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,
    FOREIGN KEY (supplier_id) REFERENCES dim_supplier(supplier_id)
);
SELECT
    *
FROM
    dim_product
```

Script Output | Query Result

SQL | All Rows Fetched: 0 in 0.055 seconds

PRODU...	PRODU...	DESCRI...	UOM	PRICE	PRODU...	SUPPLI...	ACTIV...	CREAT...	UPDAT...

- **access.dim_promotion** table creation

Worksheet | Query Builder

```
/*
create access.dim_promotion table
*/
CREATE TABLE dim_promotion (
    promotion_code VARCHAR2(6) PRIMARY KEY,
    description VARCHAR2(255),
    promotion_type VARCHAR2(10) NOT NULL,
    uom VARCHAR2(10) NOT NULL,
    promotion_value NUMBER(5, 2) NOT NULL,
    effective_from DATE,
    effective_to DATE,
    active_flag NUMBER(1, 0) DEFAULT 1 NOT NULL,
    created_timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,
    updated_timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL
);
SELECT
    *
FROM
    dim_promotion
```

Script Output | Query Result

SQL | All Rows Fetched: 0 in 0.065 seconds

PROM...	DESCRI...	PROM...	UOM	PROM...	EFFEC...	EFFEC...	ACTIV...	CREAT...	UPDAT...

- **access.dim_warehouse** table creation

Worksheet | Query Builder

```sql
/*
create access.dim_warehouse table
*/
CREATE TABLE dim_warehouse (
    warehouse_id VARCHAR2(5) PRIMARY KEY,
    address_line1 VARCHAR2(255) NOT NULL,
    address_line2 VARCHAR2(255),
    city VARCHAR2(255) NOT NULL,
    state VARCHAR2(255) NOT NULL,
    postal_code NUMBER(4, 0) NOT NULL,
    active_flag NUMBER(1, 0) DEFAULT 1 NOT NULL,
    created_timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,
    updated_timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,
    FOREIGN KEY (postal_code) REFERENCES dim_postal_code(postal_code)
);
SELECT
    *
FROM
    dim_promotion
```

Script Output | Query Result

SQL | All Rows Fetched: 0 in 0.065 seconds

PROM...	DESCRI...	PROM...	UOM	PROM...	EFFEC...	EFFEC...	ACTIV...	CREAT...	UPDAT...

- **access.dim_shipped_to** table creation

Worksheet | Query Builder

```sql
/*
create access.dim_shipped_to table
*/
CREATE TABLE dim_shipped_to (
    shipped_to_id VARCHAR2(5) PRIMARY KEY,
    customer_id VARCHAR2(10) NOT NULL,
    customer_name VARCHAR2(30) NOT NULL,
    address_line1 VARCHAR2(255) NOT NULL,
    address_line2 VARCHAR2(255),
    city VARCHAR2(255) NOT NULL,
    state VARCHAR2(255) NOT NULL,
    postal_code NUMBER(4, 0) NOT NULL,
    active_flag NUMBER(1, 0) DEFAULT 1 NOT NULL,
    created_timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,
    updated_timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,
    FOREIGN KEY (postal_code) REFERENCES dim_postal_code(postal_code)
);
SELECT
    *
FROM
    dim_shipped_to
```

Script Output | Query Result

SQL | All Rows Fetched: 0 in 0.077 seconds

SHIPPE...	CUSTO...	CUSTO...	ADDRE...	ADDRE...	CITY	STATE	POSTA...	ACTIV...	CREAT...	UPDAT...

- **access.fact_transaction_header** table creation

Worksheet | Query Builder

```sql
/*
create access.fact_transaction_header table
*/
CREATE TABLE fact_transaction_header (
    transaction_id VARCHAR2(8) PRIMARY KEY,
    "date" DATE NOT NULL,
    client_id VARCHAR2(10) NOT NULL,
    transaction_type VARCHAR2(10) NOT NULL,
    channel_id VARCHAR2(3) NOT NULL,
    payment_id VARCHAR2(6) NOT NULL,
    total_price NUMBER(5, 2) DEFAULT 0 NOT NULL,
    total_shipping_cost NUMBER(5, 2) DEFAULT 0 NOT NULL,
    payment_term_fee NUMBER(5, 2) DEFAULT 0 NOT NULL,
    total_transaction_amount NUMBER(5, 2) DEFAULT 0 NOT NULL,
    transaction_status VARCHAR2(10) NOT NULL,
    created_timestamp TIMESTAMP NOT NULL,
    updated_timestamp TIMESTAMP NOT NULL,
    FOREIGN KEY ("date") REFERENCES dim_calendar(date_id),
    FOREIGN KEY (client_id) REFERENCES dim_client(client_id),
    FOREIGN KEY (channel_id) REFERENCES dim_channel(channel_id),
    FOREIGN KEY (payment_id) REFERENCES dim_payment(payment_id)
);
SELECT
    *
FROM
    fact_transaction_header
```

Script Output | Query Result

SQL | All Rows Fetched: 0 in 0.069 seconds

TRANS...	date	CLIENT...	TRANS...	CHAN...	PAYME...	TOTAL...	TOTAL...	PAYME...	TOTAL...	TRANS...	CREAT...	UPDAT...

- **access.fact_transaction_line_items** table creation

Worksheet | Query Builder

```sql
/*
create access.fact_transaction_line_items table
*/
CREATE TABLE fact_transaction_line_items (
    line_id VARCHAR2(10) PRIMARY KEY,
    transaction_id VARCHAR2(8) NOT NULL,
    line_number NUMBER(4, 0) NOT NULL,
    product_id VARCHAR2(6) NOT NULL,
    promotion_code VARCHAR2(6),
    uom VARCHAR2(10) NOT NULL,
    quantity NUMBER(3, 0) DEFAULT 0 NOT NULL,
    price NUMBER(5, 2) DEFAULT 0 NOT NULL,
    shipping_cost NUMBER(5, 2) DEFAULT 0 NOT NULL,
    promotion_amount NUMBER(5, 2) DEFAULT 0 NOT NULL,
    total_price NUMBER(5, 2) DEFAULT 0 NOT NULL,
    warehouse_id VARCHAR2(5) NOT NULL,
    shipped_to_id VARCHAR2(5) NOT NULL,
    shipping_date DATE,
    delivered_date DATE,
    line_status VARCHAR2(10) NOT NULL,
    created_timestamp TIMESTAMP NOT NULL,
    updated_timestamp TIMESTAMP NOT NULL,
    FOREIGN KEY (transaction_id) REFERENCES fact_transaction_header(transaction_id),
    FOREIGN KEY (product_id) REFERENCES dim_product(product_id),
    FOREIGN KEY (promotion_code) REFERENCES dim_promotion(promotion_code),
    FOREIGN KEY (warehouse_id) REFERENCES dim_warehouse(warehouse_id),
    FOREIGN KEY (shipped_to_id) REFERENCES dim_shipped_to(shipped_to_id),
    FOREIGN KEY (shipping_date) REFERENCES dim_calendar(date_id),
    FOREIGN KEY (delivered_date) REFERENCES dim_calendar(date_id)
);
SELECT
    *
FROM
    fact_transaction_line_items
```

Script Output | Query Result

SQL | All Rows Fetched: 0 in 0.069 seconds

LINE_ID	TRANS...	LINE_N...	PRODU...	PROM...	UOM	QUAN...	PRICE	SHIPPI...	PROM...	TOTAL...	WAREH...	SHIPPE...	SHIPPI...	DELIVE...

SECTION B

1 Part A

Question 1

Worksheet | Query Builder

```
  GROUP BY
    country_iso_code, country_name
)
SELECT
  country_iso_code "Country ISO Code",
  country_name "Country Name",
  sales_amount sales
FROM
  sales_agg
WHERE
  rnk <= 3
ORDER BY
  sales_amount DESC
;
```

Script Output | Query Result

SQL | All Rows Fetched: 3 in 0.425 seconds

	Country ISO Code	Country Name	SALES
1	US	United States of America	52910773.15
2	DE	Germany	9210129.22
3	JP	Japan	7207880.09

Question 2

Worksheet | Query Builder

```
  GROUP BY
    calendar_year, prod_name
)
SELECT
  calendar_year,
  prod_name,
  quantity total_quantity
FROM
  sum_sold
WHERE
  rnk = 1
ORDER BY
  calendar_year
;
```

Script Output | Query Result

SQL | All Rows Fetched: 4 in 0.476 seconds

	CALENDAR_YEAR	PROD_NAME	TOTAL_QUANTITY
1	1998	O/S Documentation Set - English	4729
2	1999	Mouse Pad	4603
3	2000	1.44MB External 3.5" Diskette	4353
4	2001	Keyboard Wrist Rest	4587

Question 3

Worksheet | Query Builder

```
INNER JOIN
  sh.channels ch
ON sl.channel_id = ch.channel_id
INNER JOIN
  dim_time dt
ON sl.time_id = dt.time_id
WHERE
  prod_id = (SELECT * FROM max_prod)
GROUP BY
  prod_id, sl.channel_id, channel_desc
ORDER BY
    total_quantity DESC
;
```

Script Output | Query Result

All Rows Fetched: 3 in 0.219 seconds

	PROD_ID	CHANNEL_ID	CHANNEL_DESC	NUM_TRANS	TOTAL_QUANTITY
1	18	3	Direct Sales	1198	1198
2	18	2	Partners	666	666
3	18	4	Internet	494	494

Question 4

Worksheet | Query Builder

```
  GROUP BY
    country_iso_code, country_name
)
SELECT
  country_iso_code,
  country_name,
  sales
FROM
  sales_country
WHERE
  rnk <= 3
ORDER BY
  sales
;
```

Script Output | Query Result

All Rows Fetched: 3 in 0.194 seconds

	COUNTRY_ISO_CODE	COUNTRY_NAME	SALES
1	PL	Poland	80.97
2	NZ	New Zealand	270.93
3	CN	China	1267.91

Question 5

Worksheet | Query Builder

```
CREATE MATERIALIZED VIEW Promotion_Analysis_mv AS
  SELECT
    promo_id,
    prod_id,
    SUM(amount_sold) total_sales
  FROM
    sh.sales
  GROUP BY
    promo_id, prod_id
;
SELECT
  *
FROM
  Promotion_Analysis_mv
```

Script Output | Query Result

Fetched 50 rows in 0.069 seconds

	PROMO_ID	PROD_ID	TOTAL_SALES
1	999	19	600229.34
2	999	33	1014718.35
3	999	40	1251974.58
4	999	43	394496.38
5	999	46	230247.3

2 Part B

Question 6

Worksheet | Query Builder

```sql
SELECT
    COALESCE(TO_CHAR(promo_id), 'ALL PROMO') promo_id,
    COALESCE(TO_CHAR(prod_id), 'ALL PROD') prod_id,
    COUNT(promo_id) count_promo,
    COUNT(prod_id) count_prod,
    SUM(total_sales) total_sales
FROM
    Promotion_Analysis_mv
GROUP BY
    ROLLUP(promo_id, prod_id)
ORDER BY
    count_promo DESC
```

Script Output | Query Result

SQL | Fetched 50 rows in 0.065 seconds

	PROMO_ID	PROD_ID	COUNT_PROMO	COUNT_PROD	TOTAL_SALES
1	ALL PROMO	ALL PROD	212	212	98205831.21
2	999	ALL PROD	72	72	94504520.79
3	351	ALL PROD	71	71	1224503.26
4	350	ALL PROD	57	57	2199380.9
5	33	ALL PROD	12	12	277426.26
6	33	16	1	1	11.99
7	33	21	1	1	204297.73
8	33	26	1	1	32697.82
9	33	27	1	1	5938.68
10	33	30	1	1	2207.79
11	33	35	1	1	11097.78
12	33	40	1	1	6028.66
13	33	46	1	1	5172.75
14	33	48	1	1	1522.73
15	33	116	1	1	2398
16	33	128	1	1	4366.44
17	33	147	1	1	1685.89
18	350	13	1	1	125010.33
19	350	14	1	1	247466.94
20	350	15	1	1	27233.73
21	350	17	1	1	153137.82

With this script, management can assess the effectiveness of their promotion strategy. The data displays the number of transactions and total sales amount for each promotion code against each product ID. Additionally, it includes a summary showing the total transactions and sales amount for each promotion code across all product IDs. Since the data is sorted by the highest transaction count, management can easily view the transaction totals for all promotion codes across all products.

Question 7

Worksheet | Query Builder

```
FROM
    mtd_channel
UNION ALL
select
    calendar_year,
    CONCAT(month_desc, '-TOTAL'),
    channel_desc,
    total_mtd,
    SUM(total_mtd) OVER(PARTITION BY calendar_year order by month_desc)
FROM
    mtd
GROUP BY
    calendar_year, month_desc, channel_desc, total_mtd
ORDER BY
    calendar_year, month_desc, channel_desc
```

Script Output | Query Result

SQL | Fetched 100 rows in 0.101 seconds

	CALENDAR_YEAR	MONTH_DESC	CHANNEL_DESC	MTD_CHANNEL_SALES	YTD_CHANNEL_SALES
42	1998	1998-10-TOTAL	ALL CHANNELS	1921230.06	19978996.86
43	1998	1998-11	Direct Sales	1344301.23	14547417.97
44	1998	1998-11	Internet	332097.01	2617087.38
45	1998	1998-11	Partners	561249.46	4934585.74
46	1998	1998-11	Tele Sales	59726.93	177280.4
47	1998	1998-11-TOTAL	ALL CHANNELS	2297374.63	22276371.49
48	1998	1998-12	Direct Sales	805290.51	15352708.48
49	1998	1998-12	Internet	207879.06	2824966.44
50	1998	1998-12	Partners	391209.16	5325794.9
51	1998	1998-12	Tele Sales	50884.91	228165.31
52	1998	1998-12-TOTAL	ALL CHANNELS	1455263.64	23731635.13
53	1999	1999-01	Direct Sales	1787701.61	1787701.61
54	1999	1999-01	Internet	118625.22	118625.22
55	1999	1999-01	Partners	474131.8	474131.8
56	1999	1999-01	Tele Sales	49260.95	49260.95
57	1999	1999-01-TOTAL	ALL CHANNELS	2429719.58	2429719.58
58	1999	1999-02	Direct Sales	1562875.84	3350577.45
59	1999	1999-02	Internet	215450.1	334075.32
60	1999	1999-02	Partners	579303.32	1053435.12

This script calculates the total sales (dollars column name) amount by channel description on a monthly basis (**mtd_channel_sales**). It also calculates a rolling sum on a yearly basis (**ytd_channel_sales**), adding up the sales amounts from previous months within the current year. Additionally, it provides the total sales amount across all channels on a monthly basis. With this data, management can analyze month-on-month or year-on-year performance by each channel and determine whether each channel or the whole company's revenue met its target.